

دانشیار — سند مرجع طراحی و پیاده‌سازی

# معماری سازمانی و RBAC

از یک اپ آموزشی به یک پلتفرم آموزشی سازمان‌یافته: استان،  
ناحیه، مدرسه، کارکنان و دسترسی قابل تفویض

مهر ۱۴۰۵ | سند پایه‌ای که پنل‌های معلم، مدیر مدرسه و  
والدین از آن پیروی می‌کنند

- |                                               |                                       |
|-----------------------------------------------|---------------------------------------|
| ۰. چرا این سند، معماری پایه است               | ۸. فاعده‌ی سلسله‌مراتب تفویض          |
| ۱. سلسله‌مراتب سازمانی                        | ۹. دسترسی موقت                        |
| ۲. مدرسه به عنوان موجودیت درجه یک             | ۱۰. انتقال و عکس لحظه‌ای تاریخی       |
| ۳. مدیر ناحیه                                 | ۱۱. هرم کامل و آنچه الان ساخته می‌شود |
| ۴. هویت کارکنان از مدرسه جداست                | ۱۲. معماری بک‌اند: Modular Monolith   |
| ۵. نوع استخدام و گردش کار تأیید               | ۱۳. اثر این تغییر روی اسناد دیگر      |
| ۶. معاونین: نقش ثابت، دسترسی پویا             | ۱۴. ترتیب پیشنهادی ساخت               |
| ۷. مدل Role + Permission + Scope + Delegation | ضمیمه‌ها                              |

## ۰۰ چرا این سند، معماری پایه است

تا این جا چهار پتل (دانش‌آموز، معلم، مدیر مدرسه، والدین) طراحی شده بودند، اما دسترسی‌ها ساده فرض می‌شدند: هر کاربر یک role ثابت داشت (دانش‌آموز/معلم/مدیر/والد) و مدیر مدرسه مستقیم و دستی معلم اضافه می‌کرد. این سند آن فرض را جایگزین می‌کند با یک **مدل سازمانی واقعی**: استان، ناحیه، مدرسه به‌عنوان موجودیت‌های درجه‌یک، کارکنانی که هویتشان از مدرسه مستقل است، و دسترسی‌ای که بر پایه‌ی Role + Permission + Scope + Delegation ساخته و قابل تفویض کنترل‌شده است.

**تفاوت مهم:** این یک «ویژگی اضافه» نیست؛ معماری هسته (Core Architecture) است. اگر الان درست ساخته شود، افزودن مدیر ناحیه، مدیر استان، وزارت، مدارس غیردولتی، معاون، مشاور، انواع استخدام و انتقال کارکنان بعداً ساده است. اگر فقط role = teacher/admin/student ساده بماند، کل لایه‌ی Authorization بعداً باید از نو نوشته شود.

چهار لایه‌ای که پیش‌تر پتل‌های جدا داشتند، حالا روی یک ستون فقرات سازمانی سوار می‌شوند:

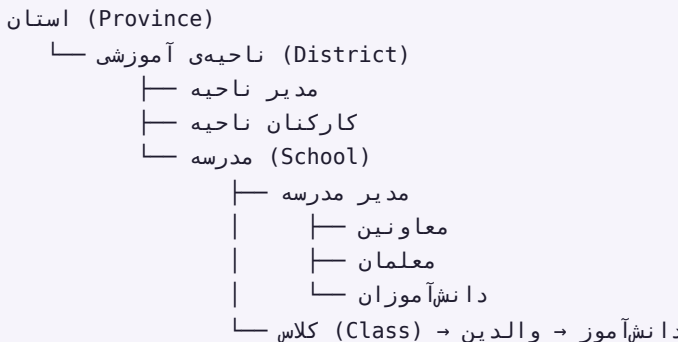
### شکل ۱ – پنج لایه‌ی دانش‌یار

سطح سازمانی: استان → ناحیه → مدرسه → کلاس  
سطح مدیریتی: مدیر ناحیه → مدیر مدرسه → معاون → معلم  
سطح آموزشی: معلم → کلاس → دانش‌آموز  
سطح خانوادگی: والد → دانش‌آموز  
Assessment → Evidence → SLM → Intervention → Retest  
سطح یادگیری: دانش‌آموز

Permission ها روی همین ساختار سوار می‌شوند، نه جدا از آن.

## ۱. سلسله مراتب سازمانی

شکل ۲ - زنجیره سازمانی



نکته کلیدی: Role (نقش، مثل «معاون») و Scope (حوزه، مثل «مدرسه ی X» یا «ناحیه ی Y») کاملاً از هم جدا هستند. دو نفر می توانند نقش یکسان «معاون» داشته باشند اما حوزه و مجوزهای کاملاً متفاوت.

## ۲. مدرسه به عنوان موجودیت درجه یک

مدرسه دیگر یک فیلد متنی داخل پروفایل کاربر نیست؛ یک موجودیت کامل در دیتابیس است.

| فیلد                           | توضیح                                     |
|--------------------------------|-------------------------------------------|
| id, name, school_code          | شناسه و کد رسمی مدرسه                     |
| school_type                    | primary   middle   high_school   combined |
| ownership_type                 | public   non_profit   private             |
| district_id, province_id       | اتصال به زنجیره سازمانی (شکل ۲)           |
| address, academic_year, status | اطلاعات عملیاتی و وضعیت فعال/غیرفعال      |

زنجیره روابط: Province → District → School → Class → Student. این ساختار پایه‌ی تحلیل‌های ناحیه‌ای و استانی در آینده است (سندهای بعدی).

### ۳. مدیر ناحیه

مدیر ناحیه یک نقش و حوزه جدید است که بین «مدیر مدرسه» و «استان» قرار می‌گیرد.

#### مدارس

- ایجاد/ثبت مدرسه
- مشاهده‌ی مدارس تحت ناحیه
- تعیین و تغییر مدیر مدرسه
- فعال/غیرفعال کردن مدرسه
- مشاهده‌ی وضعیت آموزشی مدارس (تجمیعی)

#### کارکنان ناحیه

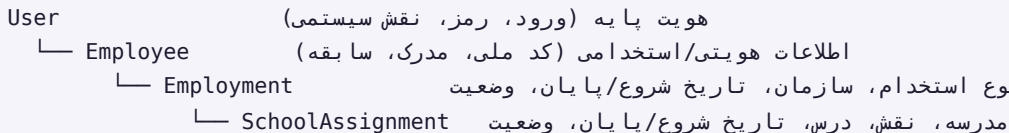
- تعریف کارکنان ناحیه
- تخصیص نقش
- مدیریت دسترسی کارکنان ناحیه

**مرز مهم:** مدیر ناحیه لزوماً به جزئیات خام دانش‌آموزان همه‌ی مدارس دسترسی ندارد؛ حوزه‌ی او (District Scope) عمدتاً داده‌ی تجمیعی و مدیریتی است، نه SLM فردی — همان اصل حداقل دسترسی که در بخش ۱۱.۱ سند دانش‌آموز برای مدیر مدرسه هم رعایت شده بود، اینجا یک پله بالاتر تکرار می‌شود.

## ۴. هویت کارکنان از مدرسه جداست

به جای ذخیره مستقیم School.teacher\_id، هویت، استخدام و تخصیص به مدرسه سه لایه جدا هستند:

### شکل ۳ — User → Employee → Employment → SchoolAssignment



مزیت کلیدی: اگر معلمی از مدرسه‌ی A به مدرسه‌ی B برود، هویتش عوض نمی‌شود؛ فقط یک SchoolAssignment جدید ساخته می‌شود و قبلی با تاریخ پایان بسته می‌شود (دقیقاً هم‌راستا با تخصیص دبیر ← کتاب ← کلاس در بخش ۳.۲ سند دانش‌آموز).

## ۵. نوع استخدام و گردش کار تأیید

نوع استخدام یک فیلد ساده نیست؛ روی آن سیاست (Policy) اجرا می‌شود.

| EmploymentType         | EmploymentOrganization        |
|------------------------|-------------------------------|
| official (رسمی)        | government   school   private |
| contractual (قراردادی) |                               |
| part_time (پاره‌وقت)   |                               |
| temporary (موقت)       |                               |

این دقیقاً همان جایی است که اصلاح درخواستی اعمال می‌شود: **مدیر مدرسه دیگر معلم را دستی و مستقیم اضافه نمی‌کند.** هر تخصیصی که با سیاست مدرسه هم‌خوان نباشد (مثلاً معلم قراردادی در یک مدرسه دولتی)، به جای ثبت مستقیم، به یک **درخواست** تبدیل می‌شود.

### شکل ۴ — از افزودن مستقیم به گردش کار تأیید



اینکه دقیقاً «چه کسی باید تأیید کند» (مدیر ناحیه، مقام مشخصی در ناحیه، یا حتی خود مدیر مدرسه برای برخی ترکیب‌ها) باید **قابل پیکربندی (Configurable)** باشد، چون مقررات مدرسه‌های مختلف (دولتی/غیرانتفاعی/خصوصی) می‌تواند متفاوت باشد — این یک جدول سیاست (Policy Table) است، نه منطق سخت‌کدشده.

## ۶. معاونین: نقش ثابت، دسترسی پویا

به جای ساختن انواع ثابت معاون (DeputyType = A/B/C/D) که سیستم را خشک می‌کند، یک نقش عمومی Deputy تعریف می‌شود و دسترسی‌هایش با چک‌لیست مجوز تعیین می‌شود.

### شکل ۵ — دو نمونه‌ی پیکربندی معاون با همان نقش

#### معاون اجرایی

- ☒ مدیریت دانش‌آموزان
- ☒ اطلاعات کلاس‌ها
- ☒ حضور و غیاب
- ☒ اطلاعات کارکنان
- ☐ مشاهده SLM جزئی
- ☐ تحلیل آموزشی پیشرفته

#### معاون آموزشی

- ☒ مشاهده عملکرد تحصیلی
- ☒ مشاهده آزمون‌ها
- ☒ تحلیل کلاس‌ها
- ☒ مشاهده معلمان
- ☐ مدیریت مالی
- ☐ مدیریت کارکنان
- ☒ مدیریت برنامه آموزشی

نقش هر دو «معاون» است؛ تفاوت فقط در Permission هاست. مدیر مدرسه هر زمان می‌تواند این چک‌لیست را تغییر دهد (بخش ۷).

## ۷. مدل Role + Permission + Scope + Delegation

دسترسی هر کاربر از ترکیب پنج جزء ساخته می‌شود:

### شکل ۶ – اجزای یک دسترسی

|            |                                                         |
|------------|---------------------------------------------------------|
| Role       | نقش (مثلاً SchoolDeputy)                                |
| Permission | مجوزهای مشخص (مثلاً view_students, analyze_assessments) |
| Scope      | حوزه (مثلاً «مدرسه‌ی شماره‌ی ۱۸۲»)                      |
| Assignment | ترکیب role+permission+scope                             |
| Delegation | اینکه چه کسی این دسترسی را تفویض کرده                   |

نمونه :

|               |                                                                    |
|---------------|--------------------------------------------------------------------|
| Role:         | SchoolDeputy                                                       |
| Permissions:  | view_students, view_classes, view_assessments, analyze_assessments |
| Scope:        | School #182                                                        |
| Delegated By: | SchoolAdmin #51                                                    |
| Valid:        | 1405/06/10 → 1406/03/30                                            |

این مدل، همان Row-Level Security و claim حوزه‌ای که در بخش ۱۱.۱ سند دانش‌آموز معرفی شد را به یک چارچوب کامل و قابل تفویض ارتقا می‌دهد؛ platform\_admin (اصلاح بخش ۰ سند معلم) هم دقیقاً یک نمونه از همین مدل است: Role=PlatformAdmin, Scope=National, با ممیزی اجباری.

## ۸. قاعده‌ی سلسله‌مراتب تفویض

**قاعده‌ی حیاتی:**  $\text{Delegated Permission} \leq \text{Owner Permission}$ . هیچ کاربری نمی‌تواند دسترسی‌ای را به دیگری تفویض کند که خودش ندارد.

مثال: اگر مدیر مدرسه خودش  $\text{Financial} = \text{No Access}$  دارد، نمی‌تواند برای معاونش  $\text{Financial} = \text{Full Access}$  فعال کند — حتی اگر از طریق رابط کاربری تلاش کند.

```
function grantPermission(grantor, grantee, permission, scope):
  if permission NOT IN effectivePermissions(grantor, scope):
    reject("شما خودتان این دسترسی را ندارید")
  else:
    create Assignment(grantee, permission, scope, delegatedBy=grantor)
    log AuditEntry(grantor, grantee, permission, scope, timestamp)
```

این قاعده باید در **بک‌اند اجرا شود، نه فقط در رابط کاربری** — یک چک صرفاً در UI با یک درخواست مستقیم به API دور زده می‌شود.

هر تغییر دسترسی (اعطا، تغییر، لغو) در لاگ ممیزی ثبت می‌شود: «مدیر مدرسه در ۱۴۰۵/۰۷/۰۸ دسترسی «مشاهده تحلیل آزمون» را برای معاون X فعال کرد.»

## ۹. دسترسی موقت

مجوزها می‌توانند زمان‌دار باشند و خودکار منقضی شوند — مثلاً معاونی که فقط تا پایان امتحانات به تحلیل آزمون دسترسی دارد:

Permission: Assessment.Analytics

Start: 1405/07/09

End: 1405/07/28

به‌صورت خودکار غیرفعال می‌شود (بدون نیاز به اقدام دستی) End, Assignment پس از →

## ۱۰. انتقال و عکس لحظه‌ای تاریخی

این بخش مستقیماً به مفهوم **Class Snapshot** (بخش ۱۲ سند مدیر مدرسه و بخش ۱۱.۲ سند دانش‌آموز) وصل می‌شود؛ انتقال معلم یا دانش‌آموز هرگز داده‌ی گذشته را تغییر نمی‌دهد.

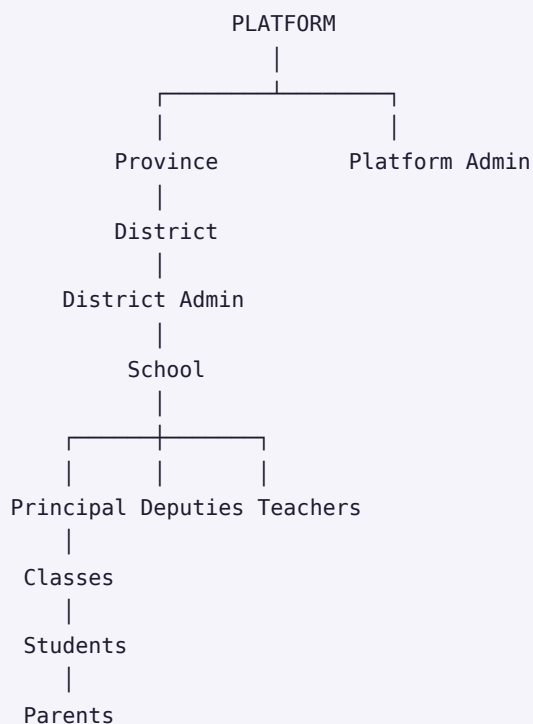
### شکل ۷ — انتقال معلم بین مدرسه‌ها

Teacher A → School A → Class ۱۰/۱ → ۱۴۰۵/۱۰/۱۰ → ۱۴۰۵/۰۶/۱۰ ریاضی  
Teacher A → School B → Class ۱۰/۲ → (فعال) → ۱۴۰۵/۱۰/۱۱ ریاضی

آزمونی که دانش‌آموز در School A داده همچنان دقیقاً می‌داند آن لحظه تحت کدام مدرسه، کلاس و معلم بوده — این برای هر تحلیل مدرسه‌ای و ناحیه‌ای بعدی ضروری است.

## ۱۱. هرم کامل و آنچه الان ساخته می‌شود

شکل ۸ — هرم کامل سازمانی (داده‌مدل آماده، پنل‌ها تدریجی)



و موازی: Student → SLM → Assessments → Activities → Errors → Interventions → Tutoring

لازم نیست همه‌ی این پنل‌ها (مدیر استان، سطح ملی) همین حالا ساخته شوند. اما دیتامدل باید همین امروز ظرفیتش را داشته باشد — یعنی جدول‌های Province و District، و فیلد province\_id/district\_id روی School، از همین فاز اول وجود دارند، حتی اگر پنل مدیر استان در فاز بعدی ساخته شود.

## ۱۲. معماری بک‌اند: Modular Monolith

با وجود گسترش قابل‌توجه مدل سازمانی، توصیه‌ی معماری این است که برای MVP سرویس‌های جدا (Microservice) ساخته نشود. پیچیدگی باید در دیتامدل، RBAC و منطق دامنه (Domain Logic) حل شود، نه در تعداد سرویس‌های مستقل.

| رویکرد           | وضعیت برای دانش‌یار                                                                                                                   |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Modular Monolith | توصیه‌شده برای MVP — مازول‌های جدا (Org, Identity/RBAC, Assessment, SLM, Tutor...) درون یک backend واحد، با مرز کد تمیز بین مازول‌ها. |
| Microservices    | فعلاً نه — پیچیدگی عملیاتی (deployment, هماهنگی داده، تراکنش بین‌سرویس) در این مرحله ارزش افزوده‌ای ندارد و توسعه را کند می‌کند.      |

این یعنی لایه‌ی جدید RBAC هم یک مازول داخل همان بک‌اند واحد است (قابل‌استفاده توسط مازول‌های Org, Assessment, SLM و غیره)، نه یک سرویس احراز هویت جدا.

## ۱۳. اثر این تغییر روی اسناد دیگر

| سند            | تغییر لازم                                                                                                                                                           |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| پنل مدیر مدرسه | بخش «افزودن معلم» باید به گردش کار درخواست/تأیید (بخش ۵) تبدیل شود؛ بخش جدید «معاونین» با چکلیست مجوز (بخش ۶) اضافه شود؛ ارجاع به مدیر ناحیه به عنوان یک سطح بالاتر. |
| پنل معلم       | بخش دسترسی/RBAC باید به مدل Role+Permission+Scope+Delegation ارجاع دهد؛ تخصیص معلم از طریق SchoolAssignment، نه فیلد ساده.                                           |
| پنل والدین     | بدون تغییر ساختاری — رابطه‌ی والد-فرزند (بخش ۱۷ سند والدین) مستقل از سلسله‌مراتب کارکنان است؛ فقط اصطلاح «سطح خانوادگی» در شکل ۱ همین سند برای هماهنگی لحاظ شد.      |

بخش بعدی هر دو سند «پنل مدیر مدرسه» و «پنل معلم» بازنگری شده و به صورت فایل‌های جدا ارسال می‌شوند.

## ۱۴. ترتیب پیشنهادی ساخت

### مرحله ۱ — دیتامدل سازمانی

- Province/District/School به عنوان جدول مستقل
- User→Employee→Employment→SchoolAssignment
- بدون پنل مدیر ناحیه هنوز.

### مرحله ۲ — RBAC هسته

- Role+Permission+Scope+Assignment · فاعده‌ی
- Delegated ≤ Owner در بک‌اند · لاگ ممیزی.

### مرحله ۳ — گردش کار استخدام

- جدول سیاست (Policy Table) · درخواست/تأیید برای ترکیب‌های
- نیازمند بررسی · کارتابل درخواست برای ناحیه.

### مرحله ۴ — معاونین و دسترسی موقت

- رابط چک‌لیست مجوز برای معاون · پشتیبانی از بازه‌ی زمانی در
- Assignment.

پنل مستقل «مدیر ناحیه» و «مدیر استان» در فازهای بعدی، پس از اثبات این لایه‌ی RBAC در سطح مدرسه، ساخته می‌شوند — دیتامدل آماده است (بخش ۱۱)، فقط رابط کاربری‌شان هنوز نیست.

## ضمیمه‌ی الف – پارامترهای پیشنهادی

| پارامتر                                      | مقدار پیش فرض                | توضیح                  |
|----------------------------------------------|------------------------------|------------------------|
| مرجع تأیید استخدام قراردادی در مدرسه‌ی دولتی | مدیر ناحیه<br>(Configurable) | بخش ۵                  |
| حداکثر بازه‌ی دسترسی موقت پیش فرض            | ۳۰ روز                       | بخش ۹؛ قابل تمدید دستی |
| سطح پیش فرض Scope مدیر ناحیه                 | District (نه School<br>فردی) | بخش ۳                  |

## ضمیمه‌ی ب – رویدادهای تحلیلی اضافه

school\_registered, district\_admin\_assigned, principal\_assigned, principal\_changed,  
employment\_request\_submitted, employment\_request\_approved, employment\_request\_rejected,  
school\_assignment\_created, school\_assignment\_transferred, deputy\_role\_created,  
permission\_granted, permission\_revoked, permission\_expired,  
delegation\_denied\_insufficient\_owner\_permission

## ضمیمه‌ی ج – تصمیم‌های باز

- فهرست دقیق ترکیب‌های (نوع مدرسه × نوع استخدام) که نیاز به تأیید ناحیه دارند، باید با مقررات رسمی آموزش و پرورش تطبیق داده شود.
- آیا مدیر ناحیه هم می‌تواند دسترسی‌هایی را به کارکنان ناحیه تفویض کند (همان قاعده‌ی بخش ۸، یک پله بالاتر)؟
- سطح استقلال مدارس غیردولتی/خصوصی در تعریف سیاست استخدام خودشان، در برابر الزامات عمومی ناحیه.